

Memory as Plans: World-Action Modeling with Memory-Grounded Planning

Sizhe Zhao¹ Haozhe Xie² Weiyu Zhao¹ Chenchu Zhang¹ Huan Wang³
Chenyang Wang¹ Qinglin Liu¹ Shengping Zhang^{† 1 4}

Project Page: [MaP-WAM](#)

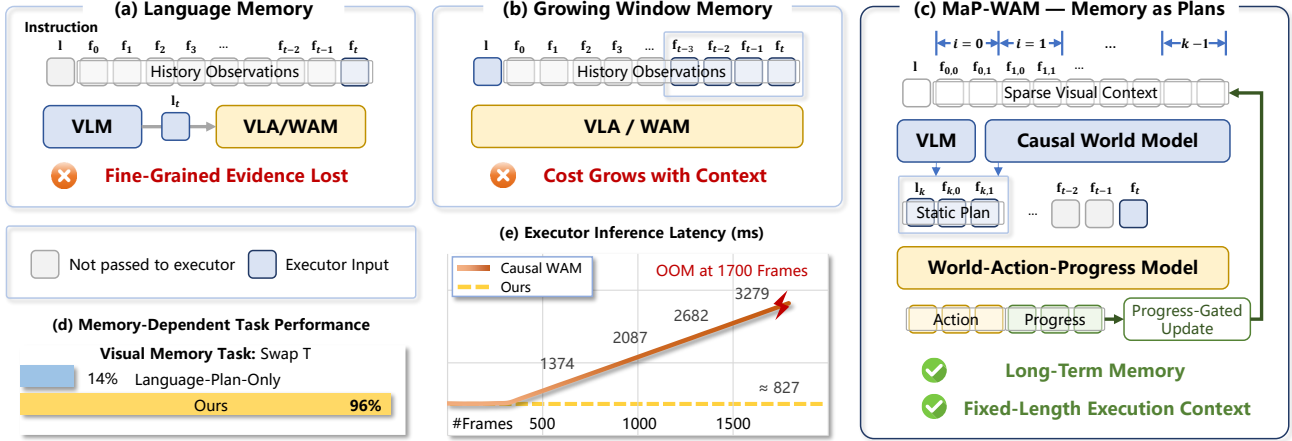


Figure 1. (a) **Language Memory** compactly summarizes past interactions but may discard **fine-grained** visual evidence, as illustrated in (d), impairing performance on tasks that require precise visual memory. (b) **Growing Window Memory** retains a window of recent observations, but extending the window to cover longer histories increases executor latency and GPU memory consumption, as shown in (e), resulting in a trade-off between history coverage and execution efficiency. (c) **MaP-WAM** constructs long-term sparse visual context by retaining a few frames from each completed segment. A vision-language model and a causal world model then convert this context into a language-visual plan. Conditioned on this static plan, the World-Action-Progress model jointly predicts action chunks and progress, enabling adaptive segment transitions and memory updates from real observations while keeping the executor context length fixed.

Abstract

Mainstream robotic policies often adopt a Markovian formulation, but many complex real-world manipulation tasks are inherently non-Markovian, requiring long-horizon memory beyond the current observation. Existing memory mechanisms often rely on language summaries, growing visual windows, or their combinations, and may therefore lose fine-grained visual evidence or face a trade-off between history coverage and execution efficiency. We introduce **MaP-WAM**, a Memory-as-Plans framework that decomposes memory-dependent world-action modeling into memory-grounded planning and plan-conditioned execution, and uses long-term multimodal episodic context as planning-time evidence rather than repeat-

edly conditioning the executor on the full history. MaP-WAM represents memory as completed segment records containing language instructions and sparse visual context, and converts this episodic memory into compact plans comprising the next segment-level language plan and corresponding visual guidance. A World-Action-Progress (WAP) model executes each plan over an unknown duration by jointly predicting action chunks and corresponding execution progress at inference time, calibrating predicted progress through plan-observation alignment for adaptive segment transitions and closed-loop context updates. MaP-WAM keeps the executor context length fixed, while structured attention further enables key-value caching in both planning and execution. MaP-WAM achieves state-of-the-art performance on RMBench with an 83.3% success rate and attains 78.0% success on real-robot tasks, while maintaining approximately constant executor inference latency as task history grows.

¹Harbin Institute of Technology, China ²Nanyang Technological University, Singapore ³Shandong University, China ⁴Harbin Institute of Technology (Weihai) Qingdao Research Institute, China. Correspondence to: Shengping Zhang <s.zhang@hit.edu.cn>.

1. Introduction

Recent advances in vision-language-action (VLA) models (Kim et al., 2024; Black et al., 2025b;a; Wang et al., 2026) and world-action models (WAMs) (Du et al., 2023; Hu et al., 2025; Kim et al., 2026; Yuan et al., 2026) have improved robotic manipulation. Yet many formulate action prediction under a Markovian assumption, treating the current observation or a fixed short history as sufficient. This approximation is inadequate for memory-dependent, partially observable tasks, where information required for a future decision may no longer be visible (Shi et al., 2026; Chen et al., 2026; Torne et al., 2026). Reliable robotic policies therefore require long-horizon memory beyond the current observation.

Existing memory mechanisms for embodied control often rely on language summaries or growing visual windows (Fig. 1(a) and (b)) (Sridhar et al., 2026; Chen et al., 2026; Torne et al., 2026; Li et al., 2026b; Ye et al., 2026; Mo-tuBrain Team et al., 2026). The former provides compact semantic abstractions but may omit fine-grained visual and spatial evidence. The latter preserves richer perceptual evidence. Causal WAMs, such as LingBot-VA (Li et al., 2026b), offer a natural mechanism for retaining long-horizon visual histories by modeling visual dynamics and actions over a growing prefix of episodic observations. However, conditioning action generation on this frame-wise history incurs increasing computational and GPU-memory costs as the context grows, creating a trade-off between inference efficiency and access to long-horizon history.

We argue that in memory-dependent tasks, long-horizon visual history is not necessarily required as a direct input to the execution model at every control step. This history is primarily needed to determine the next segment-level plan and the desired visual evolution, while execution can operate by following a memory-grounded plan. This motivates **MaP-WAM** (Fig. 1(c)), which maintains long-term memory as multimodal episodic context and uses it as planning-time evidence to generate compact plans. By decoupling memory-grounded planning from plan-conditioned execution, MaP-WAM keeps the executor context length fixed and reduces execution-time latency while preserving fine-grained grounding in long-horizon visual memory.

Concretely, MaP-WAM maintains multimodal episodic context composed of the task instruction, completed segment instructions, and sparse visual context. At each planning stage, a language planner predicts the next segment-level language plan, and a causal world model generates corresponding visual guidance conditioned on this plan and the sparse visual context. Together, the language plan and visual guidance form a memory-grounded plan that couples task semantics with anticipated visual evolution. Executing this plan, however, poses a central challenge: the required

execution duration is initially unknown, depending on task requirements and stochastic execution dynamics. We therefore introduce a World-Action-Progress (WAP) model that jointly models future visual dynamics, action chunks, and execution progress using a Mixture-of-Transformers (MoT) architecture (Liang et al., 2025). The predicted progress enables MaP-WAM to execute each plan for a variable duration and replan upon segment completion. Progress modeling also equips the executor with an explicit temporal coordinate for distinguishing visually similar observations that correspond to different semantic stages. Furthermore, the visual plan enables plan-observation alignment, which calibrates predicted progress by matching the current observation to the planned visual trajectory, thereby mitigating cumulative drift of progress prediction over long executions.

The contributions are summarized as follows:

- **Memory-as-Plans Framework:** We propose MaP-WAM, which converts long-horizon episodic evidence into memory-grounded plans, enabling fixed-context execution while retaining visual grounding.
- **Memory-Grounded Planning:** We introduce a causal world model that translates long-term visual context and the predicted segment-level language plan into a visual plan as fine-grained execution guidance.
- **Progress-Aware Execution:** We introduce WAP, which jointly models visual dynamics, action chunks, and execution progress, and combines MoT-based progress prediction with plan-observation alignment to enable variable-duration execution and adaptive segment transitions.
- MaP-WAM achieves 83.3% and 78.0% success rates on RMBench and real-robot tasks, respectively, while maintaining approximately constant executor inference latency as task history grows.

2. Related Work

2.1. Generalist Robotic Policies

Vision-Language-Action Policies. Vision-language-action (VLA) policies (Zitkovich et al., 2023; Octo Model Team et al., 2024; Kim et al., 2024; Shukor et al., 2025; Black et al., 2025b;a; Wang et al., 2026) leverage semantic priors from pretrained vision-language foundation models (Karamcheti et al., 2024; Beyer et al., 2024; Bai et al., 2025) and scale policy learning with large-scale datasets (Khazatsky et al., 2024; O’Neill et al., 2024; Bu et al., 2025), improving instruction following and task generalization. However, most VLA policies remain conditioned on the current observation or a fixed short window. Recent designs such as

DynamicVLA (Xie et al., 2026) further optimize this reactive regime for low-latency control by overlapping inference with execution. Such formulations are effective for reactive manipulation but struggle with memory-dependent tasks.

World-Action Models. World-action models (WAMs) enhance action generation through world modeling (Du et al., 2023; Hu et al., 2025; Kim et al., 2026; Yuan et al., 2026; Li et al., 2026b; Ma et al., 2026; Ye et al., 2026; MotuBrain Team et al., 2026). By predicting future latent states, future observations or action-conditioned scene evolution, WAMs provide richer learning signals than direct imitation and offer a natural interface for incorporating visual context beyond single-frame reactive control. A representative causal WAM, LingBot-VA (Li et al., 2026b), retains a growing prefix of past observations and interleaves dynamics prediction with inverse-dynamics action decoding, allowing actions to exploit all accumulated visual evidence. However, this frame-wise history incurs rapidly growing inference latency and GPU-memory costs as trajectory length increases.

Goal- and Plan-Conditioned Policies. Our work is more closely related to goal- and plan-conditioned policies, which predict intermediate goals or plans, represented as subgoal images (Zhao et al., 2025; Physical Intelligence et al., 2026), trajectories (Gu et al., 2024; Li et al., 2025b), or short videos (Du et al., 2023; Xu et al., 2025), and subsequently generate actions with plan-conditioned policies or inverse-dynamics models. However, the generated goal is usually conditioned on the current observation, task instruction, or externally provided examples rather than on accumulated episodic context, and these methods typically lack a mechanism for aligning execution progress with the generated plan. In contrast, MaP-WAM predicts memory-grounded plans and closes the loop between planning and execution through progress-aware adaptive transitions.

2.2. Memory Modeling for Robotic Manipulation

Existing works on memory modeling for robotics mainly rely on language memory, continually updated memory, growing windows, or their combinations. (1) Language Memory. This line of work converts history into a compact language summary or an intermediate instruction before action generation. MemER (Sridhar et al., 2026) and Mem-0 (Chen et al., 2026) select sparse visual keyframes and predict a subtask instruction for low-level VLA. MEM (Torne et al., 2026) maintains a recursively updated language summary as long-term memory. Such language interfaces are compact and interpretable, but may discard fine-grained perceptual evidence. (2) Continually Updated Memory. These approaches maintain updatable latent states (Li et al., 2024; 2026a) or memory banks (Fang et al., 2025; Shi et al., 2026; Manifold AI, 2026) as context for action generation, but may struggle to retain task-relevant information over long horizons, as earlier evidence can be compressed or overwrit-

ten. (3) Growing Windows. Other works directly extend the observation context through sliding or growing windows (Guhur et al., 2023; Torne et al., 2026; Chen et al., 2026; Li et al., 2025a; 2026b; Yang et al., 2026a). Direct context retains richer temporal and visual evidence, but fixed windows truncate distant history, while growing windows incur increasing latency and GPU-memory costs.

3. Our Approach

3.1. Overview

Problem Formulation. We formulate memory-dependent robotic manipulation as a sequential decision-making problem. Given a language instruction l , the current proprioceptive state s_t , and the observation sequence $f_{\leq t}$, a general memory-dependent policy models

$$\pi(a_{t+1:t+h} \mid f_{\leq t}, s_t, l), \quad (1)$$

where $a_{t+1:t+h}$ denotes the next action chunk. The key challenge is that critical information for action generation may reside in historical observations $f_{< t}$.

Memory-as-Plans Decomposition. Rather than repeatedly processing a dense, growing sequence of past observations during action generation, MaP-WAM decouples long-horizon visual-context processing from short-horizon action generation, assigning them to memory-grounded planning and plan-conditioned execution, respectively. MaP-WAM maintains a structured multimodal episodic context $C_{< k}$ before the k -th segment. This context records the execution history at the segment level, including completed segment instructions and sparsely sampled long-term visual context from previous segments. Together with the global task instruction l , $C_{< k}$ provides planning-time evidence for inferring the next language plan and the desired visual evolution. The planner models the next memory-grounded plan as

$$\pi_{\mathcal{P}}(C_k \mid C_{< k}, l). \quad (2)$$

Conditioned on the memory-grounded plan C_k , the execution module aims to predict short-horizon actions $a_{t+1:t+h}$ from the current observation f_t and robot state s_t :

$$\pi_{\mathcal{E}}(a_{t+1:t+h} \mid C_k, f_t, s_t, l), \quad [t, t+h] \subseteq \mathcal{H}(C_k), \quad (3)$$

where $\mathcal{H}(C_k)$ denotes the planning horizon of C_k , and the action timesteps $[t, t+h]$ lie within the temporal range covered by C_k . This horizon is not fixed in advance but determined online by the progress-gated segment transitions described below. $\pi_{\mathcal{E}}$ is rolled out repeatedly to generate actions within $\mathcal{H}(C_k)$ until the current plan is completed. Notably, the inputs of $\pi_{\mathcal{E}}$ are independent of the history length, so the executor context remains fixed as the task history grows.

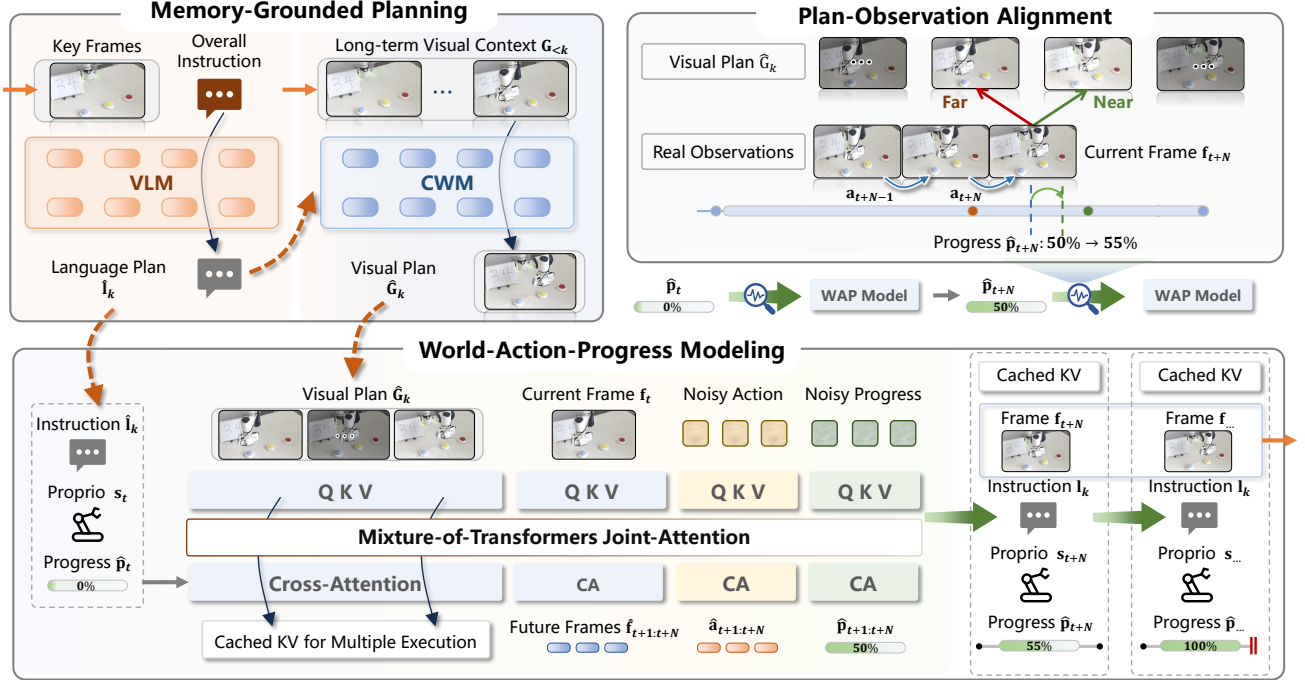


Figure 2. **Overview of MaP-WAM.** Memory-grounded planning first predicts the next segment-level language plan $\hat{l}_k$ from the multimodal context, and then generates a visual plan $\hat{G}_k$ from the long-term visual context using a causal world model (CWM) as execution guidance. Conditioned on $\hat{G}_k$ and the progress condition $\hat{p}_t$, the World-Action-Progress (WAP) model jointly models future visual dynamics, actions, and task progress. During deployment, the fixed plan prefix is cached and reused across action chunks until the predicted progress triggers the next planning stage. Plan-observation alignment further retrieves visual-plan frames near the predicted progress, matches them to the current observation, and uses the best-matching plan state to calibrate progress, mitigating error accumulation from recursive prediction over long executions. Upon segment transition, real execution observations are resampled into sparse visual evidence and appended to the episodic context.

3.2. Memory-Grounded Planning

Structured Multimodal Episodic Context. We represent execution history as a structured multimodal episodic context of completed segment records to avoid processing a dense, ever-growing sequence of past frames. Each completed segment i contributes a record $C_i = \{l_i, G_i\}$, pairing its language instruction l_i with sparse visual context G_i comprising a fixed-length sequence of frames uniformly sampled from its real execution trajectory. Additionally, the initial observation is stored as $G_0 = f_0$. The multimodal context $C_{<k}$ provides historical evidence for inferring the next language plan and desired visual evolution.

Memory-Grounded Language-Visual Planning. We factorize the planning into a language planner and a visual planner. Given the global instruction l , the completed segment instructions $l_{<k}$, and a compact keyframe set f^* extracted from $G_{<k}$ (the initial frame G_0 and the last frame of each completed segment G_i), the VLM planner $\pi_{\mathcal{P}}^l$ predicts the next subgoal as a language plan l_k :

$$\pi_{\mathcal{P}}^l(l_k | l_{<k}, l, f^*). \quad (4)$$

The resulting language plan defines the immediate semantic

objective while remaining consistent with the global instruction and completed history. We formulate $\pi_{\mathcal{P}}^v$ as a causal world model (CWM) that generates a visual plan G_k as fine-grained guidance, conditioned on the long-term visual context $G_{<k}$ of completed segments, the language plan l_k , and the global instruction l . CWM is trained with the standard flow-matching objective $\mathcal{L}_{\text{FM}}$ defined in Appendix:

$$\mathcal{L}_{\mathcal{P}}^v = \mathcal{L}_{\text{FM}}(G_k, (G_{<k}, l_k, l)), \quad (5)$$

where the generation condition and target are $c = (G_{<k}, l_k, l)$ and $y = G_k$, respectively.

Causal Attention for Visual Planning. In the CWM, the prefix comprises a variable number of blocks corresponding to the sparse visual evidence $G_{<k}$, whereas the target block represents the future guidance G_k . We organize tokens into the segment-wise blocks and apply a block-causal mask that prevents future leakage across segments and makes completed evidence a static, cacheable prefix at inference.

3.3. World-Action-Progress Modeling

Given the generated memory-grounded plan, the remaining challenge is to realize it over a variable and initially

unknown number of control steps, owing to task complexity and stochastic execution dynamics. To this end, we introduce the WAP model as a plan-conditioned executor and address the temporal misalignment through progress modeling, which provides an explicit alignment signal between the fixed plan and the evolving execution state, and enables adaptive planning-execution transitions. Unlike prior work that employs progress as a post-hoc verifier or reward signal (Zhang et al., 2025; Zhao et al., 2026), WAP treats progress as a first-class modality that is jointly generated with actions and fed back as a conditioning signal.

World-Action-Progress Model. We annotate each training segment $f_{i:j}$ with normalized progress $p_t = \frac{t-i}{j-i}$, where $t \in [i, j]$ (Zhang et al., 2025; Zhao et al., 2026). The progress value provides a continuous coordinate for aligning execution states with the visual plan G_k . As shown in Fig. 2, we construct WAP by extending a pretrained video DiT with action and progress experts in a Mixture-of-Transformers architecture to jointly model visual dynamics, robot actions, and progress. Given a segment plan $C_k = \{l_k, G_k\}$, the current observation f_t , proprioceptive state s_t , and current progress p_t , WAP jointly predicts the future visual latent, the action chunk, and the corresponding progress sequence. WAP encodes the visual plan as a static clean prefix and the current observation as clean state tokens, while appending noisy prediction targets for visual dynamics, actions, and progress. Its structured attention mask allows dynamic tokens to attend to the plan and current state, while keeping the plan prefix independent of dynamic tokens and cacheable throughout segment execution. Following FastWAM (Yuan et al., 2026), we prevent action and progress tokens from attending to future visual tokens, and vice versa. Future visual prediction therefore serves as an auxiliary world-modeling objective during training and can be omitted at inference. In cross-attention layers, all tokens attend to the language plan l_k , while dynamic tokens are additionally conditioned on the proprioceptive state s_t and current progress p_t . The progress condition p_t provides an explicit temporal anchor that disambiguates visually similar states with different semantic stages, without expanding the observation window.

Training Objective. We train WAP by applying the conditional flow-matching objective $\mathcal{L}_{\text{FM}}$ to future visual states $y_v = f_{t+1:t+h}$, an action chunk $y_a = a_{t+1:t+h}$, and a progress sequence $y_p = p_{t+1:t+h}$:

$$\mathcal{L}_{\mathcal{E}}^m = \mathcal{L}_{\text{FM}}(y_m, c), \quad m \in \{v, a, p\}, \quad (6)$$

where the shared condition is $c = (G_k, l_k, f_t, s_t, p_t)$. The final training objective is

$$\mathcal{L}_{\mathcal{E}} = \lambda_v \mathcal{L}_{\mathcal{E}}^v + \lambda_a \mathcal{L}_{\mathcal{E}}^a + \lambda_p \mathcal{L}_{\mathcal{E}}^p, \quad (7)$$

where λ_v , λ_a , and λ_p are loss weights.

3.4. Closed-Loop Planning and Execution

Plan-Observation Alignment for Progress Calibration.

WAP conditions the prediction on the current progress p_t , while ground-truth progress is unavailable at deployment. Therefore, the progress condition is recursively updated from WAP’s predicted progress sequence, causing error accumulation over long-horizon tasks. The visual plan, however, provides a temporally indexed visual reference, enabling progress calibration through plan-observation alignment. As illustrated in Fig. 2, each plan frame is indexed by normalized progress. Given the current estimate, we retrieve nearby plan frames and select the one that is visually most similar to the observation obtained after executing the current action chunk. The progress condition is updated toward the selected frame’s progress index, anchoring execution to the planned visual evolution and improving the robustness of progress estimation. See the Appendix for further details.

Progress-Gated Segment Transition. Progress prediction provides a direct criterion that enables closed-loop planning and execution. MaP-WAM averages the predicted progress over the latest actions and detects completion of the current segment once the resulting score exceeds a predefined threshold τ , terminating execution of the current plan, updating the episodic context, and invoking memory-grounded planning for the next segment. Specifically, the real execution observations are uniformly resampled into sparse visual context G_k , which replaces the generated visual plan in the appended record $\{l_k, G_k\}$, keeping the context grounded in real observations rather than generated predictions.

4. Experiments

4.1. Implementation Details

Model Configuration. For language planning, we fine-tune Qwen3.5-4B (Qwen Team, 2026) to predict the next segment-level language plan from the history. For visual planning, we initialize the CWM from WAN-2.2-5B (Wang et al., 2025) and fine-tune it using a causal input format and a block-causal attention mask. Each completed segment is uniformly resampled into $N = 8$ frames as sparse visual context. The WAP model also uses WAN-2.2 as the video expert. The action expert has 1.02B parameters with hidden dimension $d_a = 1024$, while the progress expert has 207M parameters with hidden dimension $d_p = 256$.

Training and Inference Settings. We set the threshold for planning-execution transition to $\tau = 0.95$. WAP is trained with the ground-truth G_k and l_k from the training set. The progress condition is augmented by an additive offset sampled uniformly from $[-0.1, 0.1]$ and clipped to $[0, 1]$ during training. The loss weights are $\lambda_v = \lambda_a = \lambda_p = 1.0$ for the three branches. The action and progress branches share the same sampled flow timestep, while the future

Table 1. Success rates on RMBench. Task Memory Complexity (TMC): M(1) and M(n) denote tasks requiring one and multiple task-relevant past observations, respectively. Bold and underlined entries indicate the best and second-best results.

Tasks	TMC	DP (Chi et al., 2025)	$\pi_{0.5}$ (Black et al., 2025a)	X-VLA (Zheng et al., 2025)	Mem-0 (Chen et al., 2026)	WLA-0 (Yang et al., 2026b)	LingBot-VA (Li et al., 2026b)	MaP-WAM (Ours)
Observe and Pick Up	M(1)	1%	9%	9%	4%	-	3%	19%
Rearrange Blocks	M(1)	0%	13%	13%	<u>89%</u>	-	100%	66%
Put Back Block	M(1)	0%	11%	18%	90%	-	100%	100%
Swap Blocks	M(1)	11%	24%	16%	67%	-	99%	<u>97%</u>
Swap T	M(1)	20%	15%	3%	14%	-	<u>88%</u>	96%
Battery Try	M(n)	10%	16%	26%	28%	<u>45%</u>	41%	82%
Blocks Ranking Try	M(n)	10%	6%	1%	18%	23%	100%	<u>94%</u>
Cover Blocks	M(n)	0%	0%	2%	68%	<u>84%</u>	79%	100%
Press Button	M(n)	0%	0%	0%	0%	74%	<u>84%</u>	96%
Total Average	-	5.8%	10.4%	9.8%	42.0%	-	<u>77.1%</u>	83.3%

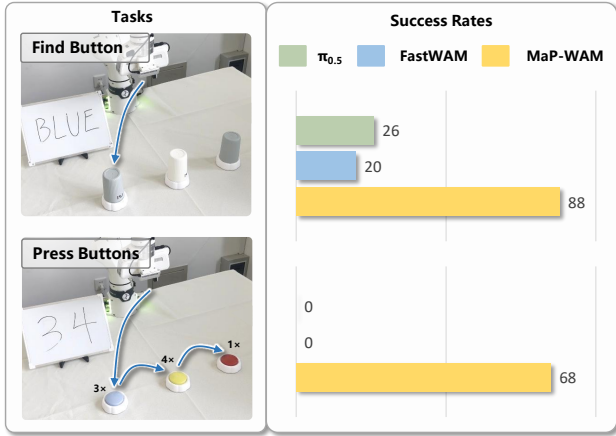


Figure 3. Real-world tasks and success rates. We report the success rates over 50 trials per task.

video branch uses an independent timestep. We use 10 flow-matching denoising steps during inference for action and progress generation. See the Appendix for further details.

4.2. Simulation Experiments

We evaluate MaP-WAM on RMBench (Chen et al., 2026), a simulation benchmark designed for long-horizon memory-dependent robotic manipulation. RMBench requires policies to reason over historical information that is no longer available from the current observation. It includes five $M(1)$ tasks and four $M(n)$ tasks, corresponding to decisions that depend on one or multiple task-relevant past observations, respectively. Following the benchmark protocol, we train MaP-WAM with 50 official expert demonstrations per task and report success rates over 100 evaluation rollouts per task with global seed 0. We compare MaP-WAM with the representative baselines evaluated on RMBench, including DP (Chi et al., 2025), $\pi_{0.5}$ (Black et al., 2025a), X-VLA (Zheng et al., 2025), Mem-0 (Chen et al., 2026), WLA-0 (Yang et al., 2026b), and LingBot-VA (Li et al., 2026b). Our

planners are trained in a multi-task setting, while WAP is trained in a single-task setting following MEM-0.

Table 1 shows that MaP-WAM achieves the highest overall success rate of 83.3% across RMBench, outperforming all representative baselines. Notably, *Swap T* and *Press Button* respectively require precise memory of initial object locations and disambiguation of visually similar but semantically distinct execution states. Benefiting from memory-grounded planning and progress modeling, MaP-WAM achieves a 96% success rate on both tasks, exceeding existing methods. Additionally, *Observe and Pick Up* is particularly challenging because it requires distinguishing among dozens of object types and jointly reasoning over temporally separated observations and spatial relations, while only 50 demonstrations are available per task. Nevertheless, MaP-WAM improves the success rate over the strongest baseline from 9% to 19%.

4.3. Real-World Experiments

Real-world experiments are conducted on a 7-DoF Franka Research 3 robot arm with third-person and wrist-mounted RealSense D435i cameras. We evaluate MaP-WAM on two categories of memory-dependent manipulation tasks. In *Find Button*, the robot first observes the colors of a set of buttons. The buttons are then occluded by covers, after which a human presents a color instruction on a whiteboard. The robot is expected to open the corresponding cover to find the button. In *Press Buttons*, the robot is expected to observe two numbers displayed on a whiteboard, press the left and middle buttons multiple times, respectively, before pressing the right confirmation button. As shown in Fig. 3, we collect 50 trajectories for each task to train these methods, and MaP-WAM achieves 88% and 68% success rates over 50 independent trials per task. By contrast, the baselines attain non-zero success on *Find Button* through random selection, but fail on the more demanding *Press Buttons* task.

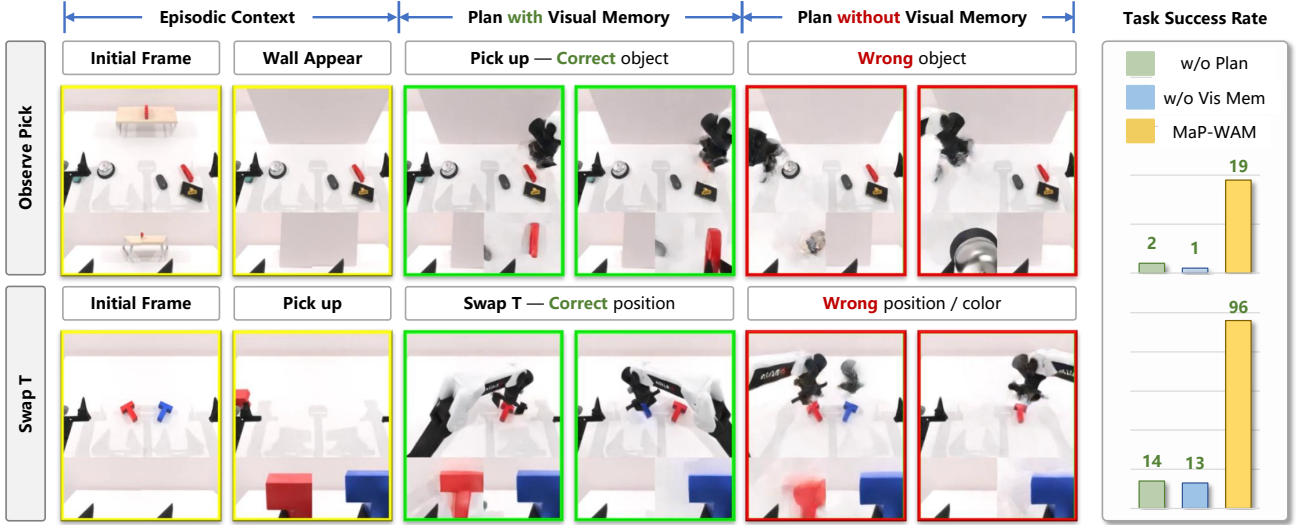


Figure 4. **Ablation study of Memory-Grounded Visual Planning.** Given the same episodic context, the CWM generates visual plans with more accurate object identities and spatial configurations. Without visual memory, a standard current-observation-conditioned world model loses fine-grained evidence and often generates plans with wrong objects, colors, or positions. Visual memory is therefore particularly important for tasks requiring precise historical visual evidence.

Table 2. **Ablation study of progress modeling designs.**

	Blocks Ranking	Cover Blocks	Press Button	Average
classification	40%	71%	0%	37.0%
w/o prog. cond.	57%	87%	18%	54.0%
w/o prog. calib.	38%	85%	98%	73.7%
MaP-WAM	94%	100%	96%	96.7%

4.4. Further Analysis

Memory-Grounded Visual Planning. We evaluate the role of visual planning on two RMBench tasks, namely *Observe and Pick Up* and *Swap T*. We compare MaP-WAM with two variants: *w/o visual plan*, which removes the CWM planner and conditions WAP only on the VLM-derived language plan, and *w/o visual memory*, which replaces the CWM with a standard current-observation-conditioned world model. The backbone architectures of all retained components are kept identical to their counterparts in MaP-WAM for a controlled comparison. As shown in Fig. 4, the *w/o visual plan* variant isolates the value of visual guidance, whereas *w/o visual memory* tests whether such guidance must be grounded in long-horizon episodic evidence rather than the current observation. Both variants reduce the success rate on *Observe and Pick Up*, showing that its performance depends on generating visual plans from historical object evidence. A similar pattern is observed on *Swap T*, where execution requires retaining the historical locations of two T-shaped blocks before swapping them. Replacing episodic visual evidence with the current observation leads to incorrect object identities or spatial positions, whereas memory-grounded

visual planning preserves critical information.

Progress Modeling. We conduct ablation experiments on progress modeling designs across three challenging RMBench tasks, namely *Blocks Ranking Try*, *Cover Blocks*, and *Press Button*. To assess the role of each design, we compare MaP-WAM with three variants: (1) *classification*, which removes progress modeling and instead adds tokens that predict whether the current segment is complete, (2) *w/o progress condition*, which retains progress prediction but removes the current progress p_t from WAP’s conditioning inputs, and (3) *w/o progress calibration*, which retains recursively predicted progress but disables plan-observation alignment at deployment. As shown in Table 2, the classification baseline yields the lowest average success rate across the three tasks. Removing the progress condition substantially degrades performance on *Press Button*. This task contains visually similar pressing and releasing phases, and without the current progress condition, the executor cannot disambiguate the current state reliably, leading to missed or repeated presses. The progress condition therefore provides an explicit temporal state signal that helps align the current observation with the intended segment execution. Progress calibration is most beneficial on *Blocks Ranking Try*, which is substantially longer than the other evaluated tasks. In particular, the subgoal *swap a and b* requires three consecutive pick-and-place operations, resulting in an average length of 423.10 steps, compared with 116.32 steps for the other tasks. The long execution horizon leads to error accumulation in autoregressive progress prediction, while progress calibration effectively mitigates this drift through plan-observation alignment.

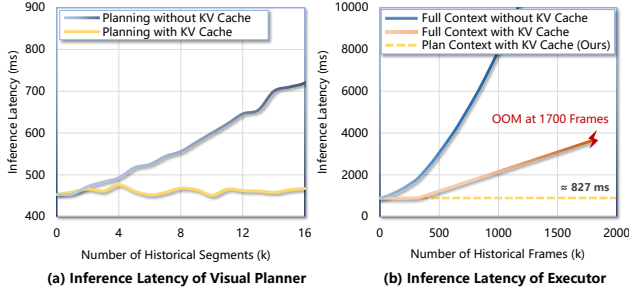


Figure 5. **Inference latency as history grows.** (a) Visual planner inference latency versus the number of completed historical segments, with and without KV caching. (b) Per-chunk executor inference latency versus the number of historical frames for full-context execution and MaP-WAM’s plan-context execution.

Inference Efficiency. Fig. 5 reports planner inference latency as the number of completed history segments increases, and executor latency per action chunk as the number of historical frames increases. For planning, MaP-WAM maintains a sparse visual context that retains only two visual latent timesteps per completed subtask after temporal compression by the WAN-VAE encoder (Wang et al., 2025). The causal attention design of the CWM makes evidence from completed segments a cacheable episodic prefix, thereby keeping planner inference efficient even after 16 consecutive segments. Planning is only invoked at each segment transition, after the preceding segment is completed. In contrast, WAP is queried repeatedly within each segment to generate action chunks. Consequently, inference latency is dominated by action generation. For execution, we compare WAP with a *Full Context* variant that conditions the same executor on the growing frame-wise observation history instead of the fixed plan prefix. Its per-chunk latency without KV caching grows rapidly as history length increases. KV caching reduces repeated prefix computation, but the full-context executor incurs roughly $4\times$ the zero-history latency with 1,500 historical frames. At 1,700 history frames, the GPU memory usage exceeds 80 GB, resulting in an out-of-memory failure. In contrast, WAP caches a fixed plan prefix, maintaining an approximately 827ms action-chunk latency across the same range.

5. Conclusion

We presented MaP-WAM for memory-dependent robotic manipulation, where task execution requires information beyond the current observation. Rather than repeatedly conditioning the executor on dense visual histories, MaP-WAM converts structured episodic context into memory-grounded plans that couple task semantics with anticipated visual evolution. By combining memory-grounded planning with progress-aware execution, MaP-WAM closes the loop among planning, execution, and context updates while preserving planning-time access to long-term, fine-grained

visual evidence and enabling efficient KV-cached inference with a fixed executor context. Experiments in simulation and on real robots show that MaP-WAM improves performance on long-horizon memory-dependent tasks while maintaining approximately constant per-chunk executor latency as task history grows.

Limitations. MaP-WAM currently builds on the segment structure available in existing benchmarks to organize memory and progress. **Extending the planning to unsegmented demonstrations via automatic segment discovery is a natural next step.** Additionally, plan-observation alignment mechanism adopts a lightweight, training-free matching metric, and learned similarity measures may further improve the calibration robustness in visually complex scenes.

References

- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al. Qwen3-VL technical report. *arXiv 2511.21631*, 2025.
- Beyer, L., Steiner, A., Pinto, A. S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschanen, M., Bugliarello, E., et al. PaliGemma: a versatile 3B VLM for transfer. *arXiv 2407.07726*, 2024.
- Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M. R., Finn, C., Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *CoRL*, 2025a.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0 : A vision-language-action flow model for general robot control. In *RSS*, 2025b.
- Bu, Q., Cai, J., Chen, L., Cui, X., Ding, Y., Feng, S., Gao, S., He, X., Hu, X., Huang, X., et al. AgiBot World Colosseum: a large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv 2503.06669*, 2025.
- Chen, T., Wang, Y., Li, M., Qin, Y., Shi, H., Li, Z., Hu, Y., Zhang, Y., Wang, K., Chen, Y., et al. RMBench: memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv 2603.01229*, 2026.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *IJRR*, 2025.
- Du, Y., Yang, S., Dai, B., Dai, H., Nachum, O., Tenenbaum, J., Schuurmans, D., and Abbeel, P. Learning universal policies via text-guided video generation. In *NeurIPS*, 2023.

- Fang, H., Grotz, M., Pumacay, W., Wang, Y. R., Fox, D., Krishna, R., and Duan, J. SAM2Act: integrating visual foundation model with a memory architecture for robotic manipulation. In *ICML*, 2025.
- Gu, J., Kirmani, S., Wohlgart, P., Lu, Y., Arenas, M. G., Rao, K., Yu, W., Fu, C., Gopalakrishnan, K., Xu, Z., Sundaresan, P., Xu, P., Su, H., Hausman, K., Finn, C., Vuong, Q., and Xiao, T. RT-Trajectory: robotic task generalization via hindsight trajectory sketches. In *ICLR*, 2024.
- Guhur, P.-L., Chen, S., Pinel, R. G., Tapaswi, M., Laptev, I., and Schmid, C. Instruction-driven history-aware policies for robotic manipulations. In *CoRL*, 2023.
- Hu, Y., Guo, Y., Wang, P., Chen, X., Wang, Y.-J., Zhang, J., Sreenath, K., Lu, C., and Chen, J. Video Prediction Policy: a generalist robot policy with predictive visual representations. In *ICML*, 2025.
- Karamcheti, S., Nair, S., Balakrishna, A., Liang, P., Kollar, T., and Sadigh, D. Prismatic VLMs: investigating the design space of visually-conditioned language models. In *ICML*, 2024.
- Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. DROID: a large-scale in-the-wild robot manipulation dataset. *arXiv 2403.12945*, 2024.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E. P., Sanketi, P. R., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., and Finn, C. OpenVLA: an open-source vision-language-action model. In *CoRL*, 2024.
- Kim, M. J., Gao, Y., Lin, T.-Y., Lin, Y.-C., Ge, Y., Lam, G., Liang, P., Song, S., Liu, M.-Y., Finn, C., and Gu, J. Cosmos Policy: fine-tuning video models for visuomotor control and planning. In *ICLR*, 2026.
- Li, H., Yang, S., Chen, Y., Chen, X., Yang, X., Tian, Y., Wang, H., Wang, T., Lin, D., Zhao, F., et al. CronusVLA: towards efficient and robust manipulation via multi-frame vision-language-action modeling. *arXiv 2506.19816*, 2025a.
- Li, H., Shen, F., Chen, D., Yang, L., Wang, X., Shi, J., Bing, Z., Liu, Z., and Knoll, A. ReMem-VLA: empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv 2603.12942*, 2026a.
- Li, L., Zhang, Q., Luo, Y., Yang, S., Wang, R., Han, F., Yu, M., Gao, Z., Xue, N., Zhu, X., et al. Causal world modeling for robot control. In *RSS*, 2026b.
- Li, X., Liu, M., Zhang, H., Yu, C., Xu, J., Wu, H., Cheang, C., Jing, Y., Zhang, W., Liu, H., et al. Vision-language foundation models as effective robot imitators. In *ICLR*, 2024.
- Li, Y., Deng, Y., Zhang, J., Jang, J., Memmel, M., Garrett, C. R., Ramos, F., Fox, D., Li, A., Gupta, A., and Goyal, A. HAMSTER: hierarchical action models for open-world robot manipulation. In *ICLR*, 2025b.
- Liang, W., Yu, L., Luo, L., Iyer, S., Dong, N., Zhou, C., Ghosh, G., Lewis, M., tau Yih, W., Zettlemoyer, L., and Lin, X. V. Mixture-of-Transformers: a sparse and scalable architecture for multi-modal foundation models. *TMLR*, 2025.
- Ma, T., Zheng, J., Wang, Z., Jiang, C., Cui, A., Liang, J., and Yang, S. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv 2603.10448*, 2026.
- Manifold AI. WorldScape Policy: generalizable robotic learning via a foundation world model. Technical report, Manifold AI, 2026.
- MotuBrain Team, Xiang, C., Bao, F., Liu, H., Tan, H., Bi, H., Li, J., Liu, J., Pang, J., Jing, K., et al. MotuBrain: an advanced world action model for robot control. *arXiv 2604.27792*, 2026.
- Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. Octo: an open-source generalist robot policy. *arXiv 2405.12213*, 2024.
- O’Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandelkar, A., Jain, A., et al. Open x-embodiment: Robotic learning datasets and RT-X models : Open x-embodiment collaboration. In *ICRA*, 2024.
- Physical Intelligence, Ai, B., Amin, A., Aniceto, R., Balakrishna, A., Balke, G., et al. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *arXiv 2604.15483*, 2026.
- Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Shi, H., Xie, B., Liu, Y., Sun, L., Liu, F., Wang, T., Zhou, E., Fan, H., Zhang, X., and Huang, G. MemoryVLA: perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *ICLR*, 2026.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv 2506.01844*, 2025.

- Sridhar, A., Pan, J., Sharma, S., and Finn, C. Scaling up memory for robotic control via experience retrieval. In *ICLR*, 2026.
- Torne, M., Pertsch, K., Walke, H., Vedder, K., Nair, S., Ichter, B., Ren, A. Z., Wang, H., Tang, J., Stachowicz, K., et al. MEM: multi-scale embodied memory for vision-language-action models. *arXiv 2603.03596*, 2026.
- Wang, A., Ai, B., Wen, B., Mao, C., Xie, C., Chen, D., Yu, F., Zhao, H., Yang, J., Zeng, J., et al. Wan: Open and advanced large-scale video generative models. *arXiv 2503.20314*, 2025.
- Wang, Y., Ding, P., Li, L., Cui, C., Ge, Z., Tong, X., Song, W., Zhao, H., Zhao, W., Hou, P., et al. VLA-Adapter: an effective paradigm for tiny-scale vision-language-action model. In *AAAI*, 2026.
- Xie, H., Wen, B., Zheng, J., Chen, Z., Hong, F., Diao, H., and Liu, Z. DynamicVLA: a vision-language-action model for dynamic object manipulation. *arXiv 2601.22153*, 2026.
- Xu, Z., Qiu, Q., and She, Y. VILP: imitation learning with latent video planning. *IEEE RA-L*, 2025.
- Yang, S., Mu, J., Wei, T., Lu, C., Li, X., Xu, L., et al. Memorywam: Efficient world action modeling with persistent memory. *arXiv 2606.20562*, 2026a.
- Yang, Y., Liu, Z., Kou, S., Chen, Y., Hu, Y., Zhou, J., Zhao, B., Wei, Z., Xia, X., Li, X., et al. World-language-action model for unified world modeling, language reasoning, and action synthesis. *arXiv 2606.05979*, 2026b.
- Ye, S., Ge, Y., Zheng, K., Gao, S., Yu, S., Kurian, G., Indupuru, S., Tan, Y. L., Zhu, C., Xiang, J., et al. World action models are zero-shot policies. *arXiv 2602.15922*, 2026.
- Yuan, T., Dong, Z., Liu, Y., and Zhao, H. Fast-WAM: do world action models need test-time future imagination? *arXiv 2603.16666*, 2026.
- Zhang, J., Luo, Y., Anwar, A., Sontakke, S. A., Lim, J. J., Thomason, J., Biyik, E., and Zhang, J. ReWiND: language-guided rewards teach robot policies without new demonstrations. In *CoRL*, 2025.
- Zhao, Q., Lu, Y., Kim, M. J., Fu, Z., Zhang, Z., Wu, Y., Li, Z., Ma, Q., Han, S., Finn, C., Handa, A., Lin, T.-Y., Wetzstein, G., Liu, M.-Y., and Xiang, D. CoT-VLA: visual chain-of-thought reasoning for vision-language-action models. In *CVPR*, 2025.
- Zhao, S., Zhang, S., Yang, S., Zhao, W., Wang, S., and Ji, X. TapSampling: inference-time sampling with a task-progress-understanding verifier for robotic manipulation. In *ICML*, 2026.
- Zheng, J., Li, J., Wang, Z., Liu, D., Kang, X., Feng, Y., Zheng, Y., Zou, J., Chen, Y., Zeng, J., et al. X-VLA: soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv 2510.10274*, 2025.
- Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohllhart, P., Welker, S., Wahid, A., et al. RT-2: vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

A. Conditional Flow Matching

For a target variable y and condition c , flow matching constructs a noisy sample by interpolating y with Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ at flow time $\rho \in (0, 1)$:

$$y^\rho = (1 - \rho)y + \rho\epsilon. \quad (8)$$

Given a velocity-field model v_θ , the standard flow-matching objective is

$$\mathcal{L}_{\text{FM}}(y, c) = \mathbb{E} \left[\|v_\theta(y^\rho, c, \rho) - (\epsilon - y)\|_2^2 \right]. \quad (9)$$

B. Construction of Multimodal Episodic Context

We represent each episode as a structured multimodal memory over segment-level records. Given an observation sequence $F = \{f_0, f_{1:i-1}, f_{i:j}, \dots\}$ partitioned into semantically coherent segments, with each segment k associated with a single language instruction l_k , we summarize each completed segment using sparse visual evidence $G_k \in \mathbb{R}^{H \times W \times 3 \times N}$. To construct G_k , we divide the observation sequence of segment k into 8 equal temporal bins and retain the last frame in each bin, yielding an ordered sequence of $N = 8$ key observations. The initial observation is retained separately as $G_0 = f_0$. During training, the annotations provided by RMBench (Chen et al., 2026) determine the segment boundaries. We define the record for each completed segment i as $C_i = (l_i, G_i)$ and the episodic context available before planning segment k as $C_{<k}$. Together with the global task instruction l , this context provides planning-time evidence for predicting the next language plan l_k and desired visual evolution G_k . For language planning, the compact keyframe set comprises G_0 and the final frame of each completed segment, while the CWM uses the full sparse visual context $G_{<k}$. Before segment k , this prefix contains $1 + 8(k - 1)$ frames and is temporally compressed by the WAN-VAE (Wang et al., 2025) into $1 + 2(k - 1)$ latent timesteps.

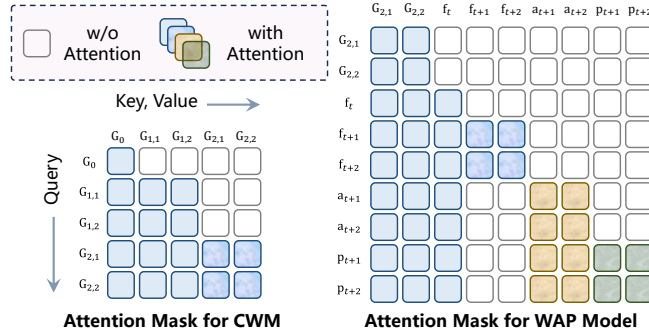


Figure 6. Self-attention masks for CWM and WAP.

C. Attention Mask and KV Caching Details

Causal Attention and KV Caching for CWM Planning. Visual evidence from completed segments $G_{<k}$ forms a causal prefix when planning segment k . As shown in Fig. 6, the block causal self-attention mask allows tokens in block i to attend only to blocks $j \leq i$. In cross-attention, completed evidence blocks attend only to the global instruction l , whereas the target block attends to the segment language plan l_k . The hidden states of the completed prefix therefore do not depend on the target block or l_k , allowing the corresponding key-value states to be computed once and reused at later planning stages within the same episode.

Plan-Prefix Caching for WAP Execution. WAP encodes the generated visual plan $\hat{G}_k$ as a static clean prefix, followed by clean state tokens for the current observation f_t and noisy target tokens for future visual latents, actions, and progress. As shown in Fig. 6, the self-attention mask allows tokens in the plan block to attend only to the plan block, whereas tokens in the state block attend to both the plan and state blocks. Future visual, action, and progress tokens attend to the plan and state blocks as well as tokens within their respective branches. Progress tokens additionally attend to action tokens, while attention between future visual tokens and action or progress tokens is masked in both directions. In cross-attention, all token groups attend to the segment-level language plan $\hat{l}_k$, whereas the state and target tokens are additionally conditioned on the proprioceptive state s_t and current progress p_t . This design allows WAP to prefill the plan prefix once per segment and reuse its cached key-value states throughout execution, as the plan representation is invariant to dynamic state and

Algorithm 1: MaP-WAM Inference

Input: Global instruction l , initial observation f_0 , and transition threshold τ
 Initialize $G_0 \leftarrow f_0$, $C_{<1} \leftarrow (G_0)$, $k \leftarrow 1$, and $t \leftarrow 0$
while the task is incomplete **do**
 Extract language planner keyframes f^* from $C_{<k}$
 Generate the language plan: $\hat{l}_k \leftarrow \pi_P^l(l, l_{<k}, f^*)$
 Generate the visual plan: $\hat{G}_k \leftarrow \pi_P^v(G_{<k}, \hat{l}_k, l)$
 Form the multimodal plan: $\hat{C}_k \leftarrow (\hat{l}_k, \hat{G}_k)$
 Prefill and cache the WAP plan prefix: $\mathcal{K}_k^E \leftarrow \pi_E(\hat{l}_k, \hat{G}_k)$
 Initialize: $p_t \leftarrow 0$, $s \leftarrow 0$, and $\mathcal{B}_k \leftarrow [f_t]$
 while $s < \tau$ **do**
 Calibrate the recurrent progress condition: $p_t \leftarrow \text{Calibrate}(f_t, \hat{G}_k, p_t)$
 Generate action and progress sequences: $(\hat{a}_{t+1:t+h}, \hat{p}_{t+1:t+h}) \leftarrow \pi_E(\mathcal{K}_k^E, f_t, s_t, p_t)$
 Execute the h predicted actions and collect observations
 Append the collected observations to $\mathcal{B}_k$
 Compute the transition score: $s \leftarrow \text{CompletionScore}(\hat{p}_{t+1:t+h})$
 Update state: $p_{t+h} \leftarrow \hat{p}_{t+h}$ and $t \leftarrow t + h$
 Resample $\mathcal{B}_k$ into N observations as visual evidence G_k
 Update episodic context: $C_{<k+1} \leftarrow \text{Append}(C_{<k}, (\hat{l}_k, G_k))$
 $k \leftarrow k + 1$

target tokens. The cache is refreshed only when a progress-gated transition triggers the generation of a new segment plan. Following FastWAM (Yuan et al., 2026), we isolate the future visual branch from the action and progress branches, allowing it to be used for auxiliary training and omitted during deployment.

D. Plan-Observation Alignment Details

At the beginning of each segment, the reference sequence comprises the initial observation of this segment and $N = 8$ generated visual plan frames, with progress increasing linearly from 0 to 1 across the resulting 9 frames. Before inferring each action chunk, we calibrate the current progress estimate p_t using the current observation f_t and the reference sequence. The two reference frames with progress values nearest to p_t are compared with f_t using mean absolute pixel difference in RGB space. The progress condition for the upcoming WAP inference is set to the average of p_t and the progress value assigned to the reference frame that is visually closer to f_t . Although not necessarily optimal for visual similarity, this metric is simple and training-free. The ablation results indicate that this lightweight calibration improves robustness to accumulated drift in recursive progress prediction over long execution horizons.

E. Algorithm Description

Algorithm 1 summarizes the overall inference procedure of MaP-WAM. At each prediction step, WAP jointly predicts an action chunk and its corresponding progress sequence. Only a prefix of the action chunk and the corresponding prefix of the progress sequence are retained for execution, progress update, and completion score computation. This truncation step is omitted from the algorithm for clarity.

F. Implementation Details

Following LingBot-VA (Li et al., 2026b), we initialize the action and progress experts by interpolating the pretrained weights of the video expert (Wang et al., 2025) to match their respective hidden dimensions and rescaling the interpolated weights to preserve output variance. For both CWM and WAP, we use 1,000 flow-matching timesteps during training and set the shift coefficient of the noise-time schedule to 5.0 during both training and inference. At inference, both CWM and WAP use 10 flow-matching denoising steps, with WAP simultaneously generating each action chunk and its corresponding progress sequence. We use an action horizon of $h = 32$ for WAP training and execute the first 8 actions at inference. All models are

Table 3. Training configurations.

Setting	VLM	CWM	WAP
Per-GPU Batch Size	4	1	8
Grad-Accumulation	2	8	8
Optimizer	AdamW	AdamW	AdamW
Learning Rate	1×10^{-4}	1×10^{-4}	1×10^{-4}
LoRA Rank	32	-	-
LoRA Alpha	64	-	-
Training Epochs	8	100	40 – 100
LR Scheduler	Cosine	Cosine	Cosine
Weight Decay	0.1	0.01	0.01

trained on eight NVIDIA A800 GPUs, each with 80 GB of memory. The number of WAP training epochs is adjusted for each task according to the number of available training frames. More training configurations are reported in Table 3.

G. Language-Planner Prompt Template

The language planner receives an ordered image sequence comprising the initial observation and the final sampled frame from each completed segment, together with the global task instruction and the corresponding segment instructions. The following is the prompt template.

Question:

<image> ... <image>

Here is an image sequence showing the process from the robot’s point of view while executing the following task: <global task instruction>.

Image 1 shows the initial environment. The completed subtask sequence is:

1. <instruction of segment 1>

2. <instruction of segment 2>

...

$k - 1$. <instruction of segment $k - 1$ >

The results after these subtasks are shown in Image 2, Image 3, ..., and Image k , respectively. Analyze what subtask k is.

Answer:

subtask k : <instruction of segment k >

H. Latency Measurement Protocol

All inference latency measurements are conducted on a single NVIDIA A800-SXM4 GPU with 80 GB of memory using PyTorch 2.7.1 and CUDA 11.8, with BF16 precision, a batch size of one, and 10 flow-matching denoising steps. The reported measurements capture latent-space model inference and exclude model loading, prompt encoding, data transfer, and VAE encoding or decoding. For each configuration, we perform five warm-up runs followed by 100 measured runs. CUDA is synchronized immediately before and after each model invocation, and the elapsed wall-clock time is averaged over the 100 measured runs. For cached inference, the prefix is prefilled before timing, so the reported latency reflects steady-state inference with an established cache. All compared configurations use the same model backbone.